

UNIVERSIDADE DO MINHO

MESTRADO EM ENGENHARIA INFORMÁTICA

Engenharia de Serviços de Redes

Grupo 13

Streaming de Áudio e Vídeo a pedido em Tempo Real

Afonso Ferreira (pg52669)

João Faria (pg53939)

Maria Cunha (pg54042)

Rui Silva (pg54213)

Setembro 2023

Conteúdo

1	Streaming HTTP simples sem adaptação dinâmica de débito	3
1.1	Capture três pequenas amostras de tráfego no link de saída do servidor, respectivamente com 1 cliente (VLC), com 2 clientes (VLC e Firefox) e com 3 clientes (VLC, Firefox e ffplay). Identifique a taxa em bps necessária (usando o ffmpeg -i videoA.mp4 e/ou o próprio wireshark), o encapsulamento usado e o número total de fluxos gerados. Comente a escalabilidade da solução. Ilustre com evidências da realização prática do exercício (ex: capturas de ecrã).	3
2	Streaming adaptativo sobre HTTP (MPEG-DASH)	7
2.1	Diga qual a largura de banda necessária, em bits por segundo, para que o cliente de streaming consiga receber o vídeo no firefox e qual a pilha protocolar usada neste cenário.	7
2.2	Ajuste o débito dos links da topologia de modo que o cliente no portátil Bela exiba o vídeo de menor resolução e o cliente no portátil Alladin exiba o vídeo com mais resolução. Mostre evidências.	8
2.3	Descreva o funcionamento do DASH neste caso concreto, referindo o papel do ficheiro MPD criado.	10
3	Streaming RTP/RTCP unicast sobre UDP e multicast com anúncios SAP	12
3.1	Compare o cenário unicast aplicado com o cenário multicast. Mostre vantagens e desvantagens na solução multicast ao nível da rede, no que diz respeito a escalabilidade (aumento do n ^o de clientes) e tráfego na rede. Tire as suas conclusões.	12
4	Conclusão	14

Lista de Figuras

1	Informação sobre o video A	3
2	Captura de um <i>packet</i> para 1 cliente	4
3	Estatística <i>Conversations</i> com 1 cliente	4
4	Estatística <i>Hierarchy</i> com 1 cliente	4
5	Estatística <i>Conversations</i> com 2 clientes	4
6	Estatística <i>Hierarchy</i> com 2 clientes	5
7	Estatística <i>Conversations</i> com 3 clientes	5
8	Estatística <i>Hierarchy</i> com 3 clientes	5
9	Informação sobre o video B de menor resolução	7
10	Informação sobre o video B de média resolução	7
11	Informação sobre o video B de alta resolução	7
12	Pilha protocolar	8
13	Topologia condicionada no <i>link</i> do portátil 'Bela'	9
14	Transmissão do vídeo com a topologia condicionada usando <i>Wireshark</i>	9
15	Criação do manifesto com todas as resoluções possíveis	10
16	Ficheiro MPD criado	10
17	Exemplo das variações entre resoluções	11
18	Topologia modificada para análise	12
19	Estatística <i>Hierarchy</i> no cenário <i>Unicast</i>	12
20	Estatística <i>Conversations</i> no cenário <i>Unicast</i>	12
21	Estatística <i>Hierarchy</i> no cenário <i>Multicast</i>	13
22	Estatística <i>Conversations</i> no cenário <i>Multicast</i>	13

1 Streaming HTTP simples sem adaptação dinâmica de débito

- 1.1 Capture três pequenas amostras de tráfego no link de saída do servidor, respetivamente com 1 cliente (VLC), com 2 clientes (VLC e Firefox) e com 3 clientes (VLC, Firefox e ffplay). Identifique a taxa em bps necessária (usando o `ffmpeg -i videoA.mp4` e/ou o próprio `wireshark`), o encapsulamento usado e o número total de fluxos gerados. Comente a escalabilidade da solução. Ilustre com evidências da realização prática do exercício (ex: capturas de ecrã).

Começamos por analisar a taxa de *bps* necessária para a transmissão do nosso vídeo (*videoA.mp4*). Para tal utilizamos o comando recomendado `ffmpeg -i videoA.mp4` e, como podemos observar pela imagem abaixo, concluímos que para a transmissão do nosso vídeo será necessária uma taxa de *bitrate* média de *12 kb/s*.

```
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'videoA.mp4':
  Metadata:
    major_brand      : isom
    minor_version    : 512
    compatible_brands: isomiso2avc1mp41
    encoder          : Lavf58.29.100
  Duration: 00:00:10.85, start: 0.000000, bitrate: 14 kb/s
    Stream #0:0(und): Video: h264 (High) (avc1 / 0x31637661), yuv420p, 200x150,
    12 kb/s, 20 fps, 20 tbr, 10240 tbn, 40 tbc (default)
    Metadata:
      handler_name    : VideoHandler
```

Figura 1: Informação sobre o video A

De seguida, analisamos um dos *packets* capturados via *wireshark* e constatamos que o encapsulamento do ficheiro decorre em todos os diversos níveis da camada protocolar. No *packet*, apresentado na figura abaixo, observamos que foi utilizado o protocolo *Ethernet* na camada de ligação de dados, na camada de rede é utilizado o protocolo *IPv4*, na camada de transporte o protocolo *TCP* e, por fim, na camada de aplicação o protocolo *HTTP*.

Analisando então os campos do *packet* recolhido, podemos afirmar que a taxa de encapsulamento neste é de 79,92% ($1210/1514 * 100$).

No.	Time	Source	Destination	Protocol	Length	Info
72	4.191362071	10.0.0.1	224.0.0.5	OSPF	78	Hello Packet
73	4.398456766	10.0.0.10	10.0.0.20	TCP	1514	8080 → 51012 [ACK] Seq=39139 Ack=1 Win=509 Len=1448 TSval=272...
74	4.398457908	10.0.0.10	10.0.0.20	TCP	1514	8080 → 51012 [ACK] Seq=40587 Ack=1 Win=509 Len=1448 TSval=272...
75	4.398458103	10.0.0.10	10.0.0.20	TCP	1341	8080 → 51012 [PSH, ACK] Seq=42035 Ack=1 Win=509 Len=1275 TSva...

▶ Frame 73: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface veth3.0.ed, id 0
 ▶ Ethernet II, Src: 00:00:00:aa:00:00 (00:00:00:aa:00:00), Dst: 00:00:00:aa:00:01 (00:00:00:aa:00:01)
 ▶ Internet Protocol Version 4, Src: 10.0.0.10, Dst: 10.0.0.20
 ▶ Transmission Control Protocol, Src Port: 8080, Dst Port: 51012, Seq: 39139, Ack: 1, Len: 1448
 ▶ [2 Reassembled TCP Segments (1719 bytes): #67(271), #73(1448)]
 ▶ Hypertext Transfer Protocol
 ▶ [truncated]000z0 00:0|0mEWU000000(0*000\035\0320000x0Y000Uo\025L0U00000000U+0U,Uc00yQQW00100|0Sj0\02300\0210UUm000\000N|00SVs0Q\
 File Data: 1210 bytes
 ▶ Data (1210 bytes)

Figura 2: Captura de um *packet* para 1 cliente

Ethernet · 3													
IPv4 · 2		IPv6 · 1		TCP · 1		UDP							
Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
10.0.0.20		51012 10.0.0.10	8080	198	135 k	99	6534	99	129 k	0.106492	9.8959		5282

Figura 3: Estatística *Conversations* com 1 cliente

Wireshark - Protocol Hierarchy Statistics - veth3.0.ed								
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	208	100.0	136505	107 k	0	0	0
Ethernet	100.0	208	2.1	2912	2283	0	0	0
Internet Protocol Version 6	1.0	2	0.1	80	62	0	0	0
Open Shortest Path First	1.0	2	0.1	72	56	2	72	56
Internet Protocol Version 4	98.1	204	3.0	4080	3199	0	0	0
Transmission Control Protocol	95.2	198	94.5	129041	101 k	179	102835	80 k
Hypertext Transfer Protocol	9.1	19	19.0	25869	20 k	19	25869	20 k
Open Shortest Path First	2.9	6	0.2	264	207	6	264	207
Address Resolution Protocol	1.0	2	0.0	56	43	2	56	43

Figura 4: Estatística *Hierarchy* com 1 cliente

Analizamos, outra vez, um *packet* recolhido entre a transmissão de vídeo com dois clientes e observamos que a taxa de encapsulamento diminui em comparação com a transmissão do vídeo a um só cliente, sendo esta 69,61% ($1054/1514 * 100$).

No.	Time	Source	Destination	Protocol	Length	Info
94	2.539195577	10.0.0.1	224.0.0.5	OSPF	78	Hello Packet
95	2.764275295	10.0.0.10	10.0.0.20	TCP	1514	8080 → 51012 [ACK] Seq=26755 Ack=1 Win=509 Len=1448 TSval=273...
96	2.764276088	10.0.0.10	10.0.0.20	TCP	1514	8080 → 51012 [ACK] Seq=28203 Ack=1 Win=509 Len=1448 TSval=273...
97	2.764276370	10.0.0.10	10.0.0.20	TCP	1514	8080 → 51012 [ACK] Seq=29651 Ack=1 Win=509 Len=1448 TSval=273...
98	2.764276654	10.0.0.10	10.0.0.20	TCP	1514	8080 → 51012 [ACK] Seq=31099 Ack=1 Win=509 Len=1448 TSval=273...
99	2.764276870	10.0.0.10	10.0.0.20	TCP	1514	8080 → 51012 [PSH, ACK] Seq=32547 Ack=1 Win=509 Len=1448 TSva...

▶ Frame 96: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface veth3.0.ed, id 0
 ▶ Ethernet II, Src: 00:00:00:aa:00:00 (00:00:00:aa:00:00), Dst: 00:00:00:aa:00:01 (00:00:00:aa:00:01)
 ▶ Internet Protocol Version 4, Src: 10.0.0.10, Dst: 10.0.0.20
 ▶ Transmission Control Protocol, Src Port: 8080, Dst Port: 51012, Seq: 28203, Ack: 1, Len: 1448
 ▶ Hypertext Transfer Protocol
 ▶ nFip0000R\001040010\035a0i Q0000\0000\03500p#00 dg00E!0008sK0fiPGR\0250\016s\024N00\030Zpyc\031?\00300:4{0000\021000+M0a0_VE0\n
 ▶ [truncated]0g0000z0q\006\0q000mI0\025C000\026-00g000\0330vv0\t0-r0A3000g00\0\035t00f+00c\026000.\02000u000000_j00:0mA000#00y,0+0X
 ▶ [truncated]0\^S|0B\022\001_00\0-(0000g0\024000g06S00\017u00x00g00;07j0\005-000\033\0020000j0)\016\00Q00u\031A+0000000PT\0360R\03310.0k0x\027m
 File Data: 1054 bytes
 ▶ Data (1054 bytes)

Wireshark - Conversations - veth3.0.ed													
Ethernet - 3		IPv4 - 3		IPv6	TCP - 2		UDP						
Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
10.0.0.20		51012 10.0.0.10	8080	136	93 k	68	4488	68	88 k	0.000000	6.0237		5960
10.0.3.20		50308 10.0.0.10	8080	136	93 k	68	4488	68	88 k	0.000038	6.0241		5960

Figura 5: Estatística *Conversations* com 2 clientes

também, sendo este não eficiente, (aumentando de $20kb/s$ com um cliente, a $57kb/s$ com dois clientes, até $80kb/s$ numa conexão com três clientes).

Por fim, analisando as estatísticas *Conversations* em cada cliente, concluímos que são respetivamente gerados 1, 2 e 3 fluxos.

2 Streaming adaptativo sobre HTTP (MPEG-DASH)

2.1 Diga qual a largura de banda necessária, em bits por segundo, para que o cliente de streaming consiga receber o vídeo no firefox e qual a pilha protocolar usada neste cenário.

```
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'videoB_200_150_200k.mp4':
Metadata:
  major_brand      : isom
  minor_version    : 512
  compatible_brands: isomiso2avc1mp41
  encoder          : Lavf58.29.100
Duration: 00:00:08.01, start: 0.000000, bitrate: 95 kb/s
Stream #0:0(und): Video: h264 (High) (avc1 / 0x31637661), yuv420p, 200x150, 91 kb/s, 29.97 fps, 29.97 tbr, 30k tbn, 59.94 tbc (default)
Metadata:
  handler_name     : VideoHandler
```

Figura 9: Informação sobre o video B de menor resolução

```
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'videoB_480_360_500k.mp4':
Metadata:
  major_brand      : isom
  minor_version    : 512
  compatible_brands: isomiso2avc1mp41
  encoder          : Lavf58.29.100
Duration: 00:00:08.01, start: 0.000000, bitrate: 276 kb/s
Stream #0:0(und): Video: h264 (High) (avc1 / 0x31637661), yuv420p, 480x360, 273 kb/s, 29.97 fps, 29.97 tbr, 30k tbn, 59.94 tbc (default)
Metadata:
  handler_name     : VideoHandler
```

Figura 10: Informação sobre o video B de média resolução

```
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'videoB_640_480_1000k.mp4':
Metadata:
  major_brand      : isom
  minor_version    : 512
  compatible_brands: isomiso2avc1mp41
  encoder          : Lavf58.29.100
Duration: 00:00:08.01, start: 0.000000, bitrate: 429 kb/s
Stream #0:0(und): Video: h264 (High) (avc1 / 0x31637661), yuv420p, 640x480, 425 kb/s, 29.97 fps, 29.97 tbr, 30k tbn, 59.94 tbc (default)
Metadata:
  handler_name     : VideoHandler
```

Figura 11: Informação sobre o video B de alta resolução

Embora tenhamos recolhido a informação acerca das três resoluções do vídeo B através do comando `ffmpeg -i videoB.mp4` [*diferentes resoluções*], estas apresentam valores que representam médias e objetivos de largura de banda possivelmente refutáveis. Deste modo, decidimos utilizar os valores presentes no ficheiro *MPD* (visível na *Figura 16*), sendo estes mais fidedignos.

Assim, a largura de banda necessária para transmitir o *videoB* gerado é de *97kb/s*, sendo esta correspondente à versão do vídeo de menor resolução. Caso fosse desejável, a largura de banda mínima necessária para transmitir o vídeo de média de resolução seria de *278kb/s*. Por fim, conseguimos concluir que, de forma a conseguirmos transmitir o vídeo com a maior resolução, seria necessário uma largura de banda de *431 kb/s*.

Os protocolos utilizados nestas transmissões foram os seguintes:

- Protocolo IP na camada de rede
- Protocolo TCP na camada de transporte
- Protocolo HTTP na camada de aplicação
- Protocolo Ethernet na camada de ligação de dados

10	10.4110158/3	10.0.3.20	10.0.0.10	TCP	66 50530 → 9999 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=34512440
11	10.411476616	10.0.3.20	10.0.0.10	HTTP	477 GET /video_dash.html HTTP/1.1
12	10.411482837	10.0.0.10	10.0.3.20	TCP	66 9999 → 50530 [ACK] Seq=1 Ack=412 Win=64768 Len=0 TSval=365119
13	10.411892829	10.0.0.10	10.0.3.20	HTTP	272 HTTP/1.1 304 Not Modified
14	10.412264641	10.0.3.20	10.0.0.10	TCP	66 50530 → 9999 [FIN, ACK] Seq=412 Ack=208 Win=64128 Len=0 TSval=365119
15	10.412270167	10.0.0.10	10.0.3.20	TCP	66 9999 → 50530 [ACK] Seq=208 Ack=413 Win=64768 Len=0 TSval=365119
16	10.879633492	10.0.3.20	10.0.0.10	TCP	74 50532 → 9999 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1
17	10.879645469	10.0.0.10	10.0.3.20	TCP	74 9999 → 50532 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
18	10.879659388	10.0.3.20	10.0.0.10	TCP	66 50532 → 9999 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=34512445
19	10.879896386	10.0.3.20	10.0.0.10	HTTP	411 GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
20	10.879901990	10.0.0.10	10.0.3.20	TCP	66 9999 → 50532 [ACK] Seq=1 Ack=346 Win=64896 Len=0 TSval=365119

▶ Frame 19: 411 bytes on wire (3288 bits), 411 bytes captured (3288 bits) on interface veth3.0.cf, id 0
▶ Ethernet II, Src: 00:00:00:aa:00:02 (00:00:00:aa:00:02), Dst: 00:00:00:aa:00:00 (00:00:00:aa:00:00)
▶ Internet Protocol Version 4, Src: 10.0.3.20, Dst: 10.0.0.10
▶ Transmission Control Protocol, Src Port: 50532, Dst Port: 9999, Seq: 1, Ack: 1, Len: 345
▶ Hypertext Transfer Protocol
 GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1\r\n
 [Expert Info (Chat/Sequence): GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1\r\n] Request Method: GET
 Request URI: /videoB_640_480_1000k_dash.mp4
 Request Version: HTTP/1.1
Host: 10.0.0.10:9999\r\nUser-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/117.0\r\nAccept: */*\r\nAccept-Language: en-US,en;q=0.5\r\nAccept-Encoding: gzip, deflate, identity\r\nRange: bytes=926-20457\r\nConnection: keep-alive\r\nReferer: http://10.0.0.10:9999/video_dash.html\r\n\r\n[Full request URI: http://10.0.0.10:9999/videoB_640_480_1000k_dash.mp4]
[HTTP request 1/1]
[Response in frame 366]

Figura 12: Pilha protocolar

2.2 Ajuste o débito dos links da topologia de modo que o cliente no portátil Bela exiba o vídeo de menor resolução e o cliente no portátil Alladin exiba o vídeo com mais resolução. Mostre evidências.

De forma a forçar o cliente no portátil 'Bela' a exibir o vídeo de menor resolução começamos por identificar que este terá que ser um valor compreendido no intervalo de $97kb/s$ inclusive a $278kb/s$, (sendo estes valores correspondentes à taxa mínima de transmissão do vídeo de menor resolução e à taxa mínima de transmissão do vídeo de média resolução, respetivamente). Assim, definiu-se um valor de $150kb/s$ para delimitar o débito no link de ligação no portátil 'Bela'. Já para o caso do cliente 'Alladin', não foi necessário fazer qualquer alteração nos links da topologia de forma a forçar a transmissão do vídeo com maior resolução uma vez que, por *default*, o vídeo de maior resolução é o que é transmitido, como podemos observar pela captura *Wireshark* anteriormente apresentada.

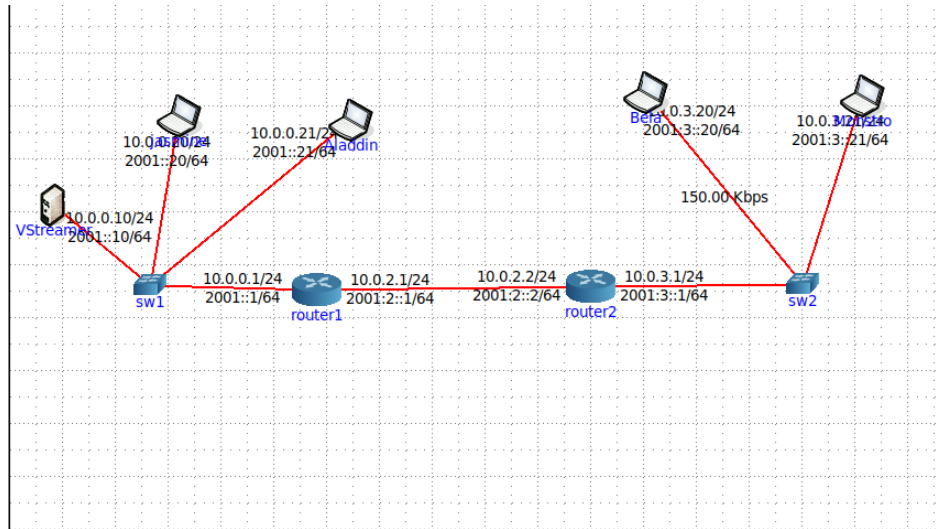


Figura 13: Topologia condicionada no *link* do portátil 'Bela'

4	0.004549878	10.0.3.20	10.0.0.10	HTTP	477	GET /video_dash.html HTTP/1.1
6	0.004700020	10.0.0.10	10.0.3.20	HTTP	272	HTTP/1.1 304 Not Modified
12	0.147825400	10.0.3.20	10.0.0.10	HTTP	505	GET /video_manifest.mpd HTTP/1.1
14	0.147958189	10.0.0.10	10.0.3.20	HTTP	273	HTTP/1.1 304 Not Modified
21	0.463311850	10.0.3.20	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
54	0.988281305	10.0.0.21	10.0.0.10	HTTP	477	GET /video_dash.html HTTP/1.1
56	0.988406922	10.0.0.10	10.0.0.21	HTTP	272	HTTP/1.1 304 Not Modified
77	1.382061810	10.0.0.21	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
398	1.388939075	10.0.0.10	10.0.0.21	MP4	567	
510	4.836187573	10.0.3.20	10.0.0.10	HTTP	412	GET /videoB_480_360_500k_dash.mp4 HTTP/1.1
1252	30.508892814	10.0.3.20	10.0.0.10	HTTP	411	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
1313	32.968492180	10.0.3.20	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
1334	33.020214978	10.0.3.20	10.0.0.10	HTTP	412	GET /videoB_480_360_500k_dash.mp4 HTTP/1.1
1511	37.251570238	10.0.0.10	10.0.3.20	MP4	670	
1761	48.010459408	10.0.3.20	10.0.0.10	HTTP	411	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
1779	48.043414239	10.0.3.20	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
1799	48.094648242	10.0.3.20	10.0.0.10	HTTP	412	GET /videoB_480_360_500k_dash.mp4 HTTP/1.1
2061	56.087988608	10.0.0.10	10.0.3.20	MP4	670	
2375	69.044368743	10.0.3.20	10.0.0.10	HTTP	411	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
2400	69.316232358	10.0.3.20	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
2420	69.368871683	10.0.3.20	10.0.0.10	HTTP	412	GET /videoB_480_360_500k_dash.mp4 HTTP/1.1
2685	77.375231813	10.0.0.10	10.0.3.20	MP4	670	

Figura 14: Transmissão do vídeo com a topologia condicionada usando *Wireshark*

Constatamos que, depois de pedir o vídeo de maior resolução (como se pode observar na linha 1313, por exemplo), este falha na sua transmissão para o portátil 'Bela'. De seguida, é pedido o vídeo de resolução média (linha 1334). Quando também este falha, uma vez que a largura de banda necessária ainda é superior à disponível no *link* de transmissão, o portátil 'Bela' acaba por pedir o vídeo de menor resolução (linha 1761).

Quanto ao portátil 'Aladdin', a transmissão na linha 77 comprova o sucesso de transmissão do vídeo de maior resolução.

2.3 Descreva o funcionamento do DASH neste caso concreto, referindo o papel do ficheiro MPD criado.

```
core@xubuncore:~$ MP4Box -dash 500 -out video_manifest videoB_200_150_200k.mp4 videoB_480_360_500k.mp4 videoB_640_480_1000k.mp4
DASH-ing files: 0.50s segments 0.50s fragments single sidx per segment
DASHing file videoB_200_150_200k.mp4
DASHing file videoB_480_360_500k.mp4
DASHing file videoB_640_480_1000k.mp4
[DASH] Generating MPD at time 2023-10-03T08:29:39.549Z
```

Figura 15: Criação do manifesto com todas as resoluções possíveis

O *DASH* (*Dynamic Adaptive Streaming Over HTTP*) é uma técnica de *streaming* de *bitrates* adaptativos, com recurso ao protocolo *HTTP*, o que possibilita a transmissão de alta qualidade de conteúdo de *media* pela Internet. O *DASH* verifica a largura de banda do *link* de ligação e avalia qual é a melhor resolução de vídeo para ser transmitido nesse mesmo link. O ficheiro MPD desempenha um papel fundamental no funcionamento do *DASH*, no nosso caso denominado "video_manifest.mpd", este ficheiro contém toda a informação sobre as várias resoluções disponíveis e sobre a largura de banda necessária para transmitir o vídeo de cada resolução específica. Durante o recorrer de *streaming* de vídeo, o *DASH* vai observando a qualidade da transmissão e vai alterando, dinamicamente, a resolução do vídeo de forma a obter a maior eficiência possível.

```
<?xml version="1.0"?>
<!-- MPD file Generated with GPAC version 0.5.2-DEV-revVersion: 0.5.2-426-gc5ad4e4+dfsg5-5 at 2023-10-03T08:29:39.549Z-->
<MPD xmlns="urn:mpeg:dash:schema:mpd:2011" minBufferTime="PT1.500S" type="static" mediaPresentationDuration="PT0H0M0.008S" maxSegmentDuration="PT0H0M0.500S" profiles="urn:mpeg:dash:profile:full:2011">
  <ProgramInformation moreInformationURL="http://gpac.sourceforge.net">
    <Title>video_manifest.mpd generated by GPAC</Title>
  </ProgramInformation>

  <Period duration="PT0H0M0.008S">
    <AdaptationSet segmentAlignment="true" bitstreamSwitching="true" maxWidth="640" maxHeight="480" maxFrameRate="30000/1001" par="4:3" lang="und">
      <SegmentList>
        <Initialization sourceURL="video_manifest_init.mp4"/>
        </SegmentList>
        <Representation id="1" mimeType="video/mp4" codecs="avc3.64000c" width="200" height="150" frameRate="30000/1001" sar="1:1" startWithSAP="0" bandwidth="97318">
          <BaseURL>videoB_200_150_200k_dash.mp4</BaseURL>
          <SegmentList timescale="30000" duration="15000">...
        </SegmentList>
        </Representation>
        <Representation id="2" mimeType="video/mp4" codecs="avc3.64001e" width="480" height="360" frameRate="30000/1001" sar="1:1" startWithSAP="0" bandwidth="278551">
          <BaseURL>videoB_480_360_500k_dash.mp4</BaseURL>
          <SegmentList timescale="30000" duration="15000">...
        </SegmentList>
        </Representation>
        <Representation id="3" mimeType="video/mp4" codecs="avc3.64001e" width="640" height="480" frameRate="30000/1001" sar="1:1" startWithSAP="0" bandwidth="431368">
          <BaseURL>videoB_640_480_1000k_dash.mp4</BaseURL>
          <SegmentList timescale="30000" duration="15000">...
        </SegmentList>
        </Representation>
      </AdaptationSet>
    </Period>
  </MPD>
```

Figura 16: Ficheiro MPD criado

Podemos verificar, na figura abaixo, uma variação de resoluções aquando da transmissão do vídeo através de um *link* com uma largura de banda limitada. Isto acontece pois o *DASH* verifica que a transmissão na rede não era a indicada para o vídeo com maior resolução e, por isso, decidiu transmitir um vídeo com uma resolução mais baixa, mais adequada à largura de banda do *link* de conexão.

http						
No.	Time	Source	Destination	Protocol	Length	Info
5	1.302500708	10.0.0.10	10.0.0.10	HTTP	477	GET /video_dash.html HTTP/1.1
7	1.302631086	10.0.0.10	10.0.0.10	HTTP	272	HTTP/1.1 304 Not Modified
13	1.543613105	10.0.0.10	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
97	3.494061466	10.0.0.10	10.0.0.10	HTTP	410	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
430	13.789587720	10.0.0.10	10.0.0.10	MP4	670	
757	29.053710765	10.0.0.10	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
1273	49.438486707	10.0.0.10	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
1811	70.787027668	10.0.0.10	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
2392	207.598625878	10.0.0.10	10.0.0.10	HTTP	477	GET /video_dash.html HTTP/1.1
2394	207.598776919	10.0.0.10	10.0.0.10	HTTP	272	HTTP/1.1 304 Not Modified
2400	207.855171682	10.0.0.10	10.0.0.10	HTTP	411	GET /videoB_640_480_1000k_dash.mp4 HTTP/1.1
2488	209.966767518	10.0.0.10	10.0.0.10	HTTP	410	GET /videoB_200_150_200k_dash.mp4 HTTP/1.1
2823	220.257050789	10.0.0.10	10.0.0.10	MP4	670	

Figura 17: Exemplo das variações entre resoluções

3 Streaming RTP/RTCP unicast sobre UDP e multicast com anúncios SAP

3.1 Compare o cenário unicast aplicado com o cenário multicast. Mostre vantagens e desvantagens na solução multicast ao nível da rede, no que diz respeito a escalabilidade (aumento do nº de clientes) e tráfego na rede. Tire as suas conclusões.

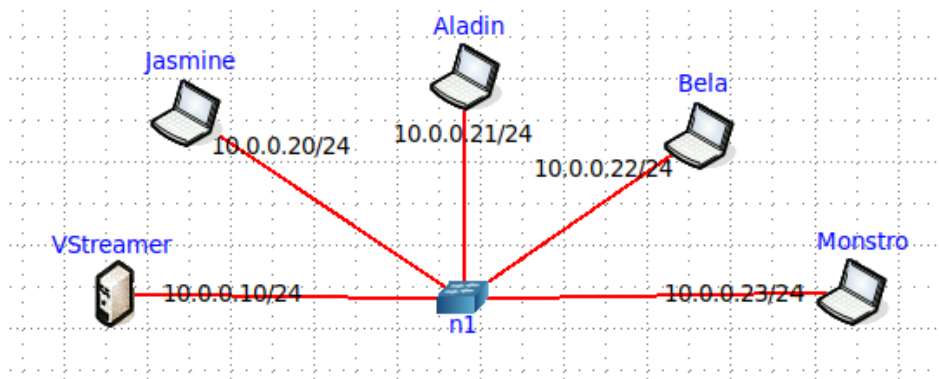


Figura 18: Topologia modificada para análise

O cenário *unicast*, que consiste numa conexão envolvendo apenas um servidor e um cliente, também denominada de 'conexão um para um', apresenta uma implementação muito simples, uma vez que apenas temos uma conversa a decorrer. A utilização de uma conexão 'um para um' oferece mais precisão e controlo na transmissão de dados mas é, no entanto, mais ineficiente em termos de largura de banda. Esta solução tem uma grande desvantagem que é o facto de não ser escalável, isto é, o seu funcionamento é gravemente prejudicado quantos mais clientes se conectarem à rede.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	824	100.0	591947	155 k	0	0	0
Ethernet	100.0	824	1.9	11536	3026	0	0	0
Internet Protocol Version 6	0.1	1	0.0	40	10	0	0	0
Internet Control Message Protocol v6	0.1	1	0.0	16	4	1	16	4
Internet Protocol Version 4	99.6	821	2.8	16420	4308	0	0	0
User Datagram Protocol	99.6	821	1.1	6568	1723	0	0	0
Data	98.8	814	94.0	556303	145 k	814	556303	145 k
ADwin configuration protocol	0.8	7	0.2	1008	264	7	1008	264
Address Resolution Protocol	0.2	2	0.0	56	14	2	56	14

Figura 19: Estatística *Hierarchy* no cenário *Unicast*

Ethernet · 2		IPv4 · 1		IPv6 · 1		TCP		UDP · 2											
Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A						
10.0.0.10	40651	10.0.0.23	5555	815	591 k	815	591 k	0	0	0.000000	30.4899	155 k							
10.0.0.10	40652	10.0.0.23	5556	6	420	6	420	0	0	1.088696	25.1503	133							

Figura 20: Estatística *Conversations* no cenário *Unicast*

O cenário *multicast* consiste numa conexão 'um para vários', o que significa que um só servidor consegue atender a vários clientes em simultâneo. As grandes vantagens no uso deste cenário é a escalabilidade desta abordagem assim como a eficiência de utilização da largura de banda necessária, que neste caso, analisando as estatísticas, concluímos ser de *119kb/s*, sendo esta significativamente menor do que a requerida num cenário *unicast* para fazer cada transmissão, sendo esta, neste outro caso, de *145kb/s*. No entanto, este cenário também apresenta desvantagens, nomeadamente a complexidade na sua configuração e gestão de grupos *multicast*, como também a falta de garantia na entrega dos dados aos clientes. Alguns clientes podem não receber certos ou todos os pacotes em transmissão na rede caso estes não estejam configurados corretamente ou se houver problemas na rede.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s
Frame	100.0	779	100.0	551807	153 k	0	0	0
Ethernet	100.0	779	2.0	10906	3034	0	0	0
Internet Protocol Version 6	0.1	1	0.0	40	11	0	0	0
Internet Control Message Protocol v6	0.1	1	0.0	16	4	1	16	4
Internet Protocol Version 4	99.9	778	2.8	15560	4330	0	0	0
User Datagram Protocol	99.9	778	1.1	6224	1732	0	0	0
Session Announcement Protocol	0.6	5	0.3	1620	450	0	0	0
Session Description Protocol	0.6	5	0.3	1500	417	5	1500	417
Real-Time Transport Protocol	83.2	648	79.5	438784	122 k	0	0	0
MP4V-ES	83.2	648	78.1	431008	119 k	648	431008	119 k
Real-time Transport Control Protocol	0.6	5	0.0	140	38	5	140	38
Data	15.4	120	14.2	78517	21 k	120	78517	21 k

Figura 21: Estatística *Hierarchy* no cenário *Multicast*

Ethernet · 4		IPv4 · 2		IPv6 · 2		TCP		UDP · 3									
Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A				
10.0.0.10	33433	224.0.0.200	5555	2,152	1565 k	2,152	1565 k	0	0	0.000000	80.5492	155 k					
10.0.0.10	34321	224.2.127.254	9875	16	5856	16	5856	0	0	4.400583	75.5498	620					
10.0.0.10	33434	224.0.0.200	5556	16	1120	16	1120	0	0	4.503410	75.4990	118					

Figura 22: Estatística *Conversations* no cenário *Multicast*

Adicionalmente, ao analisar as estatísticas retiradas em ambos cenários, observamos que num cenário *multicast* existe, como esperado, uma maior perda de *packets* quando comparado ao cenário *unicast*.

Concluindo, o cenário *multicast* é mais eficiente em termos de largura de banda e escalabilidade, tornando-o mais adequado para distribuir dados para um grande número de clientes. Já o cenário *unicast* é mais preciso e oferece controlo granular, mas é ineficiente ao nível do uso de largura de banda e não é escalável. A escolha entre *multicast* ou *unicast* depende das necessidades específicas de uma rede. Em cenários em que a largura de banda é crítica e a escalabilidade é importante, *multicast* é preferível. No entanto, em cenários em que a entrega de dados deve ser precisa e o controlo da transmissão destes é fundamental, então um cenário *unicast* pode ser a melhor opção.

4 Conclusão

Com este trabalho prático foi nos possível explorar várias técnicas e vários protocolos associados ao *streaming* de ficheiros de vídeo. Para diferentes cenários de *streaming* foi possível fazer uma análise sobre o tráfego e a escalabilidade dos mesmos na rede, adquirindo conhecimentos sobre o funcionamento do *DASH* que nos permitiu observar, dinamicamente, alterações na resolução do vídeo a ser transmitido, consoante a largura de banda disponível.

Por fim, pudemos ainda comparar cenários *unicast* com cenários *multicast* e perceber as vantagens e as desvantagens de cada um.

Finalizando, o grupo encontra-se satisfeito com o trabalho realizado pois alcançou e respondeu a todas as questões propostas pelos docentes, justificando adequadamente as mesmas.